

Lab 7-2

Shared Memory

TA: Kuan-Tsen Liu

Prof.: Hsung-Pin Chang

$ Outline

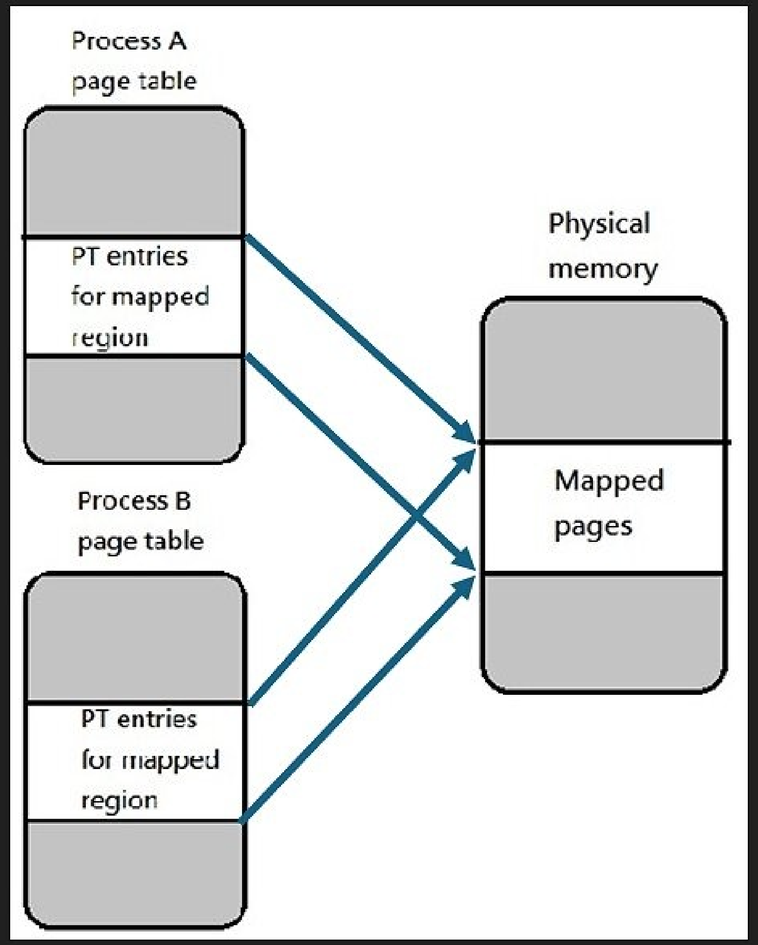
- Shared Memory
  - Overview
  - API
  - Example2
  - Exercise2

$ Overview (1/6)

- Two techniques that allow unrelated processes to share memory regions for IPCs
  - shared memory-mapped file
    - Need to explicitly create a file
  - Shared memory
    - Without needing to create a corresponding mapped file

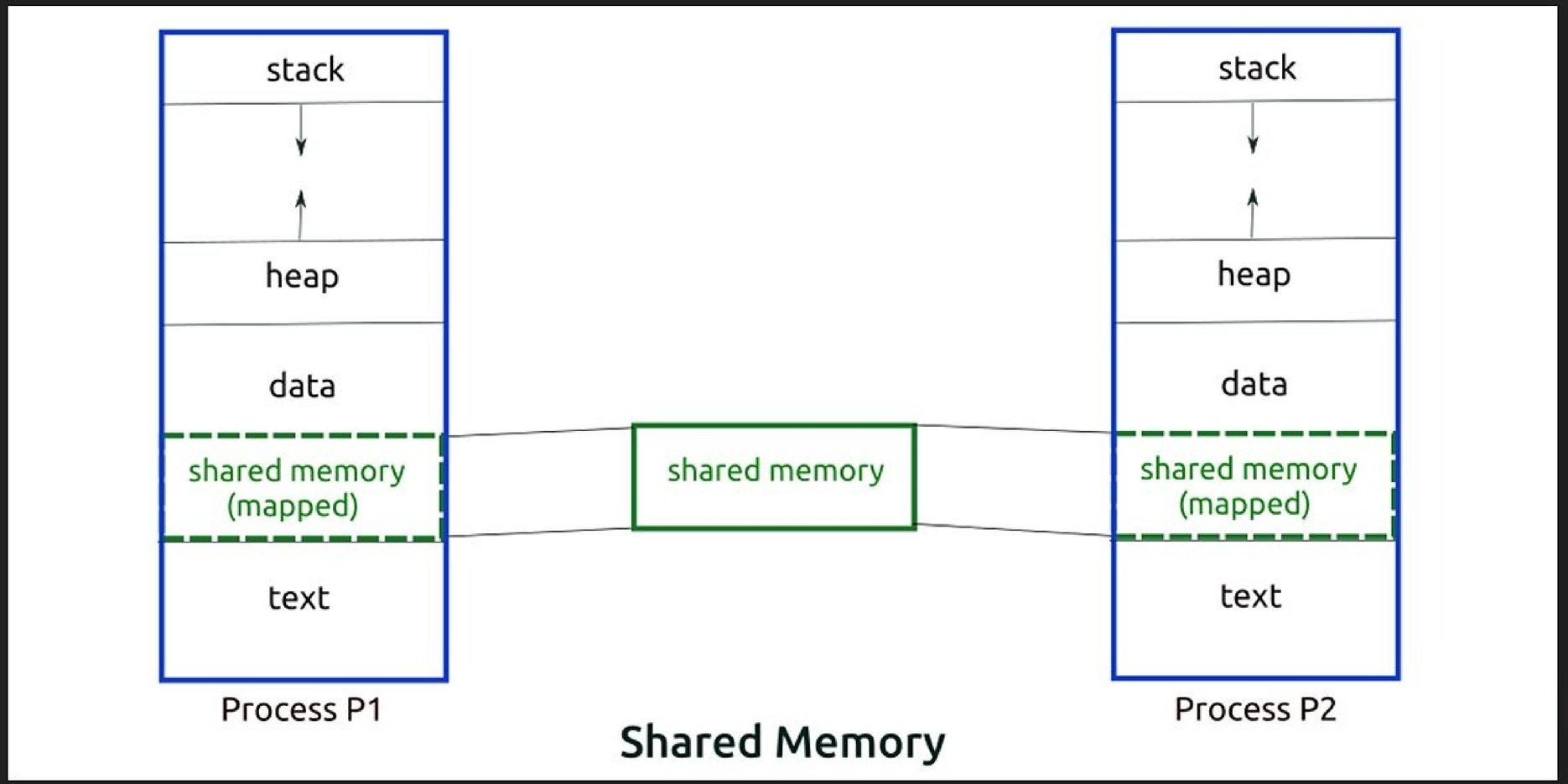
$ Overview (2/6)

- Shared memory-mapping file



$ Overview (3/6)

- Locations of Shared Memory



$ Overview (4/6)

- POSIX Shared Memory
  - To use a POSIX shared memory object
  - Use the shm_open() function to open or create an object with a specified name
  - Return a file descriptor referring to the object
  - Analogous to the open() system call
  - Pass the file descriptor in a call to mmap()
  - Specifies MAP_SHARED in the flags argument

$ Overview (5/6)

- POSIX Shared Memory
  - Linux create a pseudo-file for shared memory :
    - objects in a special location /dev/shm
  - The shared memory has kernel persistence :
    - The objects will persist even if no process currently has them open
      - Until the system is shut down or calling unlink()

$ Overview (6/6)

- Similar to the memory-mapped files
  - Use a semaphore (or other synchronization primitive) to synchronize their accesses, if two or more processes access a memory-mapped file simultaneously
  - Introduced later in Lab 15

$ Outline

- Shared Memory
  - Overview
  - API
  - Example2
  - Exercise2

$ API (1/12)

- POSIX API

Function	Description
shm_open()	Creates and opens a new shared memory object or opens an existing object.
ftruncate()	Set the size of a file to the value specified by length.
mmap()	Creates a new mapping in the calling process's virtual address space.
munmap()	Removing a mapping from the calling process's virtual address space.
shm_unlink()	Removes the shared memory object specified by name.

$ API (2/12)

- shm_open() : Creating Shared Memory Objects

```
#include<sys/mman.h>
int shm_open(const char *name, int oflag, mode_t mode);
EX: shm_open( NAME, O_CREAT|O_EXCL, 0666 );
```

- Returns file descriptor on success, or –1 on error
- name : Identifies the shared memory object to be created or opened.
- oflag : Mask of bits that modify the behavior of the call.
- mode : File Permission

$ API (3/12)

- shm_open() oflag argument :
  - Library fcntl.h

Flag	Description
O_CREAT *O_EXCL*	Create object if it doesn't already exist With O_CREAT, create object exclusively
O_RDONLY *O_RDWR*	Open for read-only access Open for read-write access
O_TRUNC	Truncate object to zero length

$ API (4/12)

- shm_open() mode argument :
  - Library sys/stat.h
  - Constants for File Permission Bits

Constant	Octal value	Permission bit
S_IRUSR	0400	User-read
S_IWUSR	0200	User-write
S_IXUSR	0100	User-execute
S_IRGRP	040	Group-read
S_IWGRP	020	Group-write
S_IXGRP	010	Group-execute
S_IROTH	04	Other-read
S_IWOTH	02	Other-write
S_IXOTH	01	Other-execute

$ API (5/12)

- mmap() :

```
#include<sys/mman.h>
void *mmap(void *addr, size_t length, int prot, int flags,
                int fd, off_t offset);
EX: mmap(NULL, 1024, PROT_WRITE, MAP_SHARED, shm_fd, 0);
```

- Returns starting address of mapping on success or MAP_FAILED on error

$ API (6/12)

```
void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

- addr : Indicates the virtual address at which the mapping is to be located.
- length : Specifies the size of the mapping in bytes.
- prot : A bit mask specifying the protection to be placed on the mapping.

$ API (7/12)

```
void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

- flags : A bit mask of options controlling various aspects of the mapping operation.
- fd : A file descriptor identifying the file to be mapped.
- offset : Specifies the starting point of the mapping in the file.

$ API (8/12)

- Memory protection values **prot** argument :

Value	Description
PROT_READ	The contents of the region can be read
PROT_WRITE	The contents of the region can be modified
PROT_EXEC	The contents of the region can be executed
PROT_NONE	The region can not be accessed

$ API (9/12)

- Bit-mask values for the mmap() flags argument :

Value	Description	SUSv3
MAP_ANONYMOUS	Create an anonymous mapping	
MAP_FIXED	Interpret *addr* argument exactly (Section 49.10)	•
MAP_LOCKED	Lock mapped pages into memory (since Linux 2.6)	
MAP_HUGETLB	Create a mapping that uses huge pages (since Linux 2.6.32)	
MAP_NORESERVE	Control reservation of swap space (Section 49.9)	
MAP_PRIVATE	Modifications to mapped data are private	•
MAP_POPULATE	Populate the pages of a mapping (since Linux 2.6)	
MAP_SHARED	Modifications to mapped data are visible to other processes and propagated to underlying file (converse of MAP_PRIVATE)	•
MAP_UNINITIALIZED	Don't clear an anonymous mapping (since Linux 2.6.33)	

$ API (10/12)

- ftruncate() :

```
#include<unistd.h>
int ftruncate(int fildes, off_t length);
EX: ftruncate( fd, 2048);
```

- Return 0 on success, or -1 on error
- fildes : File descriptor.
- length : Size of the file.

$ API (11/12)

- munmap() : Removing Shared Memory Objects

```
#include<sys/mman.h>
int munmap(void *addr, size_t length);
EX: munmap( address, 2048);
```

- Return 0 on success, or –1 on error
- addr : The starting address of the address range to be unmapped.
- length : The size (in bytes) of the region to be unmapped.

$ API (12/12)

- shm_unlink() : Removing Shared Memory Objects

```
#include<sys/mman.h>
int shm_unlink(const char *name);
EX: shm_unlink( NAME );
```

- Return 0 on success, or -1 on error
- name : Identifies the shared memory object to be removed.

$ Outline

- Shared Memory
  - Overview
  - API
  - Example2
  - Exercise2

$ producer

```c
#include<fcntl.h>
#include<stdio.h>
#include<sys/mman.h>
#include<unistd.h>    // ftruncate() & close()

#define NAME "/myShareMemory"
#define SMSIZE 1024

int main() {
    int fd = shm_open(NAME, O_CREAT | O_RDWR, 0666);  // file descriptor
    if (fd == -1) {
        perror("Failed to open or create the shared memory. \n");
        return 1;
    }

    if (ftruncate(fd, SMSIZE) == -1) {
        perror("Failed to resize. \n");
        return 1;
    }

    void *map_addr = mmap(NULL, 1024, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (map_addr == MAP_FAILED) {
        perror("Failed to create the map. \n");
        return 1;
    }

    char *ptr, buffer[SMSIZE];
    ptr = map_addr;
    sprintf(ptr, "%s", buffer);

    if (munmap(map_addr, 1024) == -1) {
        perror("Failed to unmap. \n");
        return 1;
    }

    if (close(fd) == -1) {
        perror("Failed to close the shared memory. \n");
        return 1;
    }

    return 0;
}
```

$ consumer

```c
#include<fcntl.h>
#include<stdio.h>
#include<sys/mman.h>

#define NAME "/myShareMemory"
#define SMSIZE 1024

int main() {
    int fd = shm_open(NAME, O_RDONLY, 0666);
    if (fd == -1) {
        perror("Failed to open or create the shared memory. \n");
        return 1;
    }

    void *map_addr = mmap(NULL, 1024, PROT_READ, MAP_SHARED, fd, 0);
    if (map_addr == MAP_FAILED) {
        perror("Failed to read the map. \n");
        return 1;
    }
    printf("%p\n", map_addr);

    return 0;
}
```

$ Outline

- Shared Memory
  - Overview
  - API
  - Example2
  - Exercise2

$ Exercise 2 (50 pts.) (1/3)

- Write two programs:
  - producer.c
    - create a shared memory
    - user can type some message into the shared memory
  - consumer.c
    - print the context of shared memory on the screen

$ Exercise 2 (2/3)

- Compiler & Linking
  - On Linux, programs employing the POSIX IPC mechanisms must be linked with the librt Library
  - Specifying the -lrt option to the gcc command.

```
amanda@oslab:~/lab6_2$ gcc -o producer producer.c -lrt
amanda@oslab:~/lab6_2$ gcc -o consumer consumer.c -lrt
```

$ Exercise 2 (3/3)

- Result



$ Reference

- *The Linux Tutorial*
- https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_shared_memory.htm
- https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_message_queues.htm
  - *The Linux Programmer's Guide*
  - *http://tldp.org/LDP/lpg/*



Thanks for your listening

Made by Jerry Liu in 2026/10